

Vision 3일차 과제 보고서

머신러닝 개념 및 PyTorch 대응 정리

학번 : 2026406076

이름 : 정창규

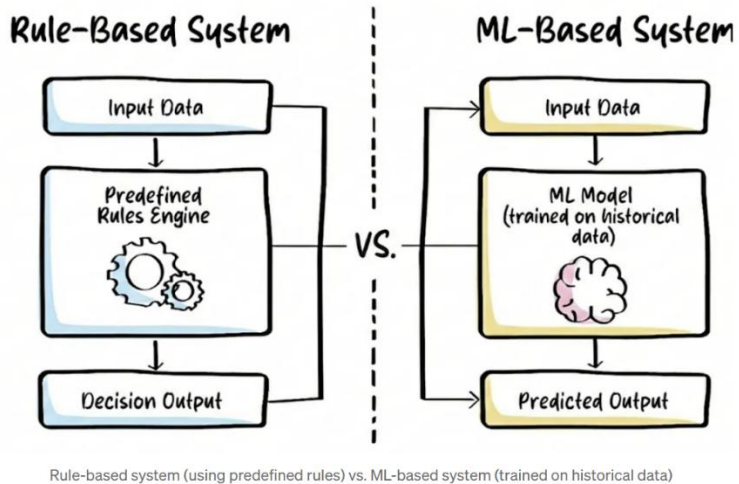
제출일 : 2026-09-28

목차

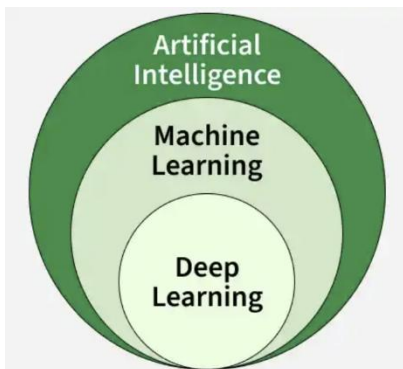
1. 머신러닝의 개념
2. 머신러닝의 종류
 - 2.1 지도학습
 - 2.2 비지도학습
 - 2.3 강화학습
3. 퍼셉트론
 - 3.1 퍼셉트론의 학습
4. 활성화 함수
 - 4.1 대표적인 활성화 함수
5. 비용 함수
 - 5.1 MSE와 MAE
 - 5.2 경사하강법
6. 딥러닝 및 심층신경망
 - 6.1 순전파
7. 역전파 알고리즘
 - 7.1 신경망 학습의 전체 흐름
8. PyTorch 대응 정리
 - 8.1 퍼셉트론과 nn.Linear
 - 8.2 활성화 함수
 - 8.3 비용 함수
 - 8.4 경사하강법과 Optimizer
 - 8.5 역전파와 Autograd
 - 8.6 심층신경망
9. PyTorch를 이용한 신경망 학습 예제
10. 학습에 사용되는 주요 설정값
 - 10.1 Epoch
 - 10.2 Batch와 Batch Size
 - 10.3 Learning Rate
 - 10.4 Optimizer
11. 머신러닝 개념과 PyTorch 대응 정리
12. 결론 및 느낀점

1. 머신러닝의 개념

머신러닝은 사람이 문제를 해결하기 위한 규칙을 직접 작성하는 대신, 데이터를 이용하여 모델이 규칙이나 패턴을 학습하도록 하는 방법이다. 기존의 규칙 기반 방식에서는 사람이 조건과 규칙을 코드로 직접 작성하고, 프로그램이 입력 데이터에 해당 규칙을 적용하여 결과를 출력한다. 반면 머신러닝에서는 데이터를 이용해 모델을 학습하고, 학습된 모델에 새로운 데이터를 입력하여 추론을 수행한 뒤 결과를 얻는다.



위 그림에서 확인할 수 있듯이 규칙 기반 방식은 사람이 미리 정의한 규칙을 입력 데이터에 적용하여 결과를 결정한다. 반면 머신러닝 기반 방식은 데이터를 이용하여 학습된 모델이 입력 데이터를 처리하고 예측 결과를 출력한다. 따라서 머신러닝에서는 사람이 모든 판단 규칙을 직접 작성하기보다, 데이터로부터 규칙을 학습할 수 있도록 모델을 구성하는 것이 중요하다.



또한 인공지능, 머신러닝, 딥러닝은 서로 포함 관계를 가진다. 인공지능은 가장 넓은 개념이며, 그 안에 머신러닝이 포함되고 머신러닝의 한 분야로 딥러닝이 존재한다.

$$AI \supset ML \supset DL$$

2. 머신러닝의 종류

머신러닝은 학습 과정에서 정답을 어떤 방식으로 제공하는지에 따라 크게 지도학습, 비지도학습, 강화학습으로 구분할 수 있다. 지도학습은 입력 데이터와 정답을 함께 제공하고, 비지도학습은 정답 없이 데이터만 제공하며, 강화학습은 정답 대신 행동에 대한 보상을 제공한다.

2.1 지도학습

지도학습은 입력 데이터와 정답에 해당하는 레이블을 함께 제공하여 모델을 학습시키는 방식이다. 모델은 입력 데이터와 정답 사이의 관계를 학습하고, 이후 새로운 데이터가 입력되었을 때 적절한 결과를 예측한다.

지도학습의 예로는 YOLO를 이용한 공과 골대 검출이 있으며, 대표적인 문제로 분류와 회귀가 있다.

2.2 비지도학습

비지도학습은 정답에 해당하는 레이블을 제공하지 않고 데이터만을 이용하여 학습하는 방식이다. 모델은 데이터 내부에 존재하는 구조나 특징을 스스로 찾아내며, 비슷한 특성을 가진 데이터를 묶거나 데이터의 구조를 분석한다.

대표적인 방법으로 군집화와 차원 축소가 있으며, K-means를 이용하여 비슷한 특성을 가진 데이터를 여러 그룹으로 구분하는 방법이 이에 해당한다.

2.3 강화학습

강화학습은 정답을 직접 제공하는 대신 행동에 대한 보상을 이용하여 학습하는 방식이다. 모델은 여러 행동을 반복적으로 시도하면서 보상이 더 커지는 방향으로 학습하며, 이를 통해 상황에 적합한 행동을 익히게 된다. 강화학습의 예로는 휴머노이드의 보행이 있으며, 관련된 학습 방법으로 정책 학습과 가치 학습이 있다.



3. 퍼셉트론

퍼셉트론은 생물의 뉴런을 모방하여 만들어진 기본적인 인공신경망 모델이다. 1957년 로젠블라트가 제안하였으며, 여러 개의 입력값을 받아 각 입력값에 가중치를 곱한 뒤 모두 더하고, 여기에 편향을 추가하여 하나의 값을 계산한다.

퍼셉트론의 기본 계산식은 다음과 같다.

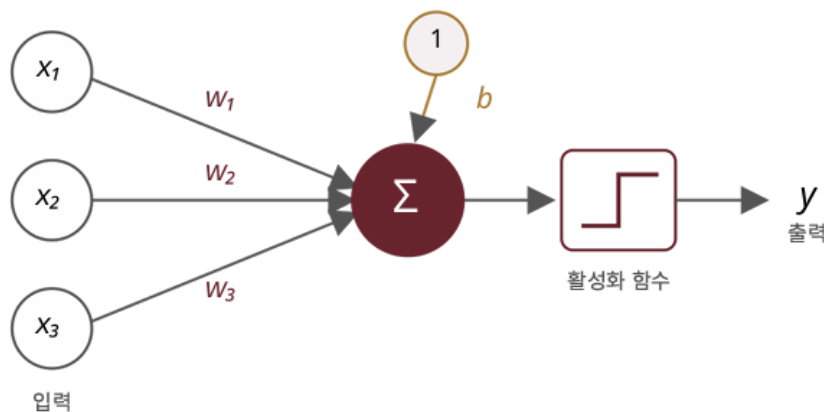
$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

여기서 x_1, x_2, x_3 는 입력값, w_1, w_2, w_3 는 각 입력에 대한 가중치, b 는 편향을 의미한다. 가중치는 각 입력이 결과에 얼마나 중요한 영향을 주는지를 나타내며, 편향은 출력값이 결정되는 기준을 조절하는 역할을 한다.

계산된 z 는 활성화 함수를 통과하여 최종 출력값으로 변환된다. 초기 퍼셉트론에서는 계단 함수를 사용하여 다음과 같이 출력값을 결정한다.

$$y = \begin{cases} 1 & (z \geq 0) \\ 0 & (z < 0) \end{cases}$$

즉, z 가 0 이상이면 1을 출력하고, 0보다 작으면 0을 출력한다. 이러한 가중치와 편향은 학습 과정에서 조정된다.



3.1 퍼셉트론의 학습

퍼셉트론은 예측한 결과와 실제 정답을 비교하고, 예측이 잘못된 경우 가중치를 수정하는 방식으로 학습한다.

$$w \leftarrow w + \eta(y - \hat{y})x$$

여기서 y 는 실제 정답, $\hat{y}$ 는 퍼셉트론이 예측한 값, η 는 학습률을 의미한다. 학습률은 한 번의 학습 과정에서 가중치를 얼마나 크게 수정할지를 결정한다.

예측값이 정답과 같으면 $y - \hat{y} = 0$ 이 되므로 가중치는 변하지 않는다. 반대로 정답과 예측값이 다르면 정답과 예측값의 차이, 입력값, 학습률에 따라 가중치를 수정한다. 이러한 과정을 반복하면서 더 적절한 가중치를 찾아간다.

4. 활성화 함수

활성화 함수는 뉴런에서 계산된 가중합 z 를 입력으로 받아 다음 층으로 전달할 출력값을 결정하는 함수이다.

$$a = f(z)$$

활성화 함수의 가장 중요한 역할은 신경망에 비선형성을 추가하는 것이다. 만약 활성화 함수가 선형 함수라면 여러 개의 층을 쌓더라도 전체 계산은 하나의 선형 연산으로 표현할 수 있기 때문에 깊은 신경망을 구성하는 의미가 줄어들는다. 반대로 비선형 활성화 함수를 사용하면 층을 거칠 때마다 입력을 다양한 형태로 변환할 수 있어 복잡한 데이터의 특징이나 결정 경계를 표현할 수 있다.

4.1 대표적인 활성화 함수

대표적인 활성화 함수에는 Step, Sigmoid, Tanh, ReLU, Leaky ReLU가 있다. 각 함수는 출력 범위와 기울기의 특성이 다르기 때문에 신경망의 목적과 구조에 따라 선택하여 사용한다.

Step 함수는 입력값이 기준 이상이면 1을 출력하고, 기준보다 작으면 0을 출력하는 함수이다.

$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

초기 퍼셉트론에서 사용되는 단순한 형태의 활성화 함수이지만, 대부분의 구간에서 미분값이 0이기 때문에 경사하강법을 이용한 학습에는 적합하지 않다.

Sigmoid 함수는 다음과 같이 표현된다.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

출력값이 0과 1 사이에 존재하기 때문에 확률처럼 해석할 수 있다는 특징이 있다. 하지만 입력값의 절댓값이 커질수록 기울기가 0에 가까워져 학습이 잘 이루어지지 않는 기울기 소실 문제가 발생할 수 있다.

Tanh 함수는 출력값이 -1과 1 사이에 존재하며, Sigmoid와 달리 출력의 중심이 0이라는 특징이 있다. 하지만 입력값이 매우 크거나 작아질 경우 기울기 소실이 발생할 수 있다. ReLU 함수는 다음과 같이 표현된다.

$$f(z) = \max(0, z)$$

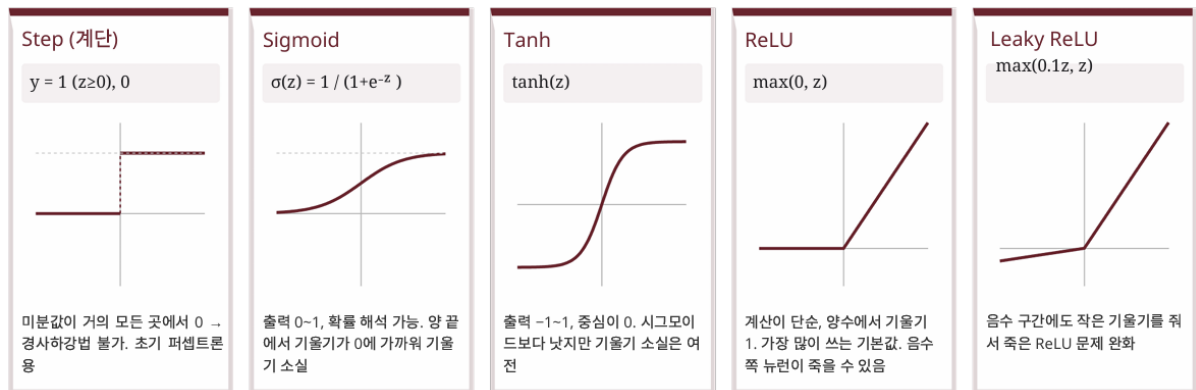
입력값이 음수이면 0을 출력하고, 양수이면 입력값을 그대로 출력한다. 계산이 단순하고 양수 구간에서 기울기가 1이기 때문에 많이 사용되는 활성화 함수이다. 그러나 음수 구간에서는 출력과 기울기가 모두 0이 되기 때문에 일부 뉴런이 더 이상 학습되지 않는 문제가 발생할 수 있다.

Leaky ReLU 함수는 ReLU의 음수 구간에도 작은 기울기를 남겨 두도록 만든 함수이다.

$$f(z) = \max(0.1z, z)$$

음수 구간에서도 작은 값이 출력되기 때문에 ReLU에서 발생할 수 있는 죽은 뉴런 문제를 완화할

수 있다.



5. 비용 함수

비용 함수는 모델의 예측이 실제 정답과 얼마나 다른지를 전체 데이터 또는 배치를 기준으로 하나의 수치로 나타내는 함수이다. 모델의 예측이 정답에 가까울수록 비용 함수의 값은 작아지고, 정답과 차이가 클수록 값은 커진다. 신경망 학습의 목표는 이러한 비용 함수의 값을 최소화할 수 있는 가중치를 찾는 것이다.

손실 함수와 비용 함수는 비슷한 의미로 사용되지만 적용 범위에서 차이가 있다. 손실 함수(Loss)는 하나의 데이터에 대한 예측 오차를 나타내며, 비용 함수(Cost)는 전체 데이터 또는 하나의 배치에서 계산된 손실의 평균을 의미한다.

회귀 문제에서는 실제값과 예측값이 수치적으로 얼마나 차이가 나는지를 계산하며, 대표적인 방법으로 MSE와 MAE가 있다. 분류 문제에서는 모델이 정답 클래스에 어느 정도의 확률을 부여했는지를 이용하며 Binary Cross Entropy와 Categorical Cross Entropy 등이 사용된다. 객체 검출에서는 하나의 손실만 사용하는 것이 아니라 박스의 위치, 클래스, 객체성 등에 대한 여러 손실을 함께 사용한다.

회귀 문제 정답이 숫자 → 얼마나 멀리 떨어졌나 MSE MAE	분류 문제 정답이 클래스 → 정답 클래스에 확률을 얼마나 줬나 Binary CE Categorical CE	객체 검출 (YOLO) 여러 손실을 합쳐서 사용 박스 위치 클래스 객체성
---	--	---

5.1 MSE와 MAE

회귀 문제에서는 예측값과 실제값 사이의 차이를 측정하기 위해 MSE와 MAE를 사용할 수 있다. 두 방법 모두 오차의 크기를 계산하지만 오차를 처리하는 방식에는 차이가 있다.

MSE(Mean Squared Error)는 평균 제곱 오차로, 실제값과 예측값의 차이를 제곱한 뒤 전체 데이터에 대해 평균을 계산한다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

오차를 제곱하기 때문에 큰 오차가 발생할수록 비용 함수의 값이 크게 증가한다. 또한 미분이 매끄럽다는 특징이 있어 학습 과정에서 많이 사용된다. 하지만 큰 오차에 큰 영향을 받기 때문에 이상치에 민감하다는 단점이 있다.

MAE(Mean Absolute Error)는 평균 절대 오차로, 실제값과 예측값의 차이에 절댓값을 적용한 뒤 평균을 계산한다.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE는 오차를 제곱하지 않고 절댓값을 사용하기 때문에 MSE보다 이상치의 영향을 적게 받는다. 하지만 오차가 0인 지점에서는 미분이 불가능하다는 특징이 있다.

평균 제곱 오차 (MSE)

$$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$$

오차를 제곱 → 큰 오차에 벌을 크게 준다. 미분이 매끄러워 가장 많이 쓰임.
대신 이상치에 민감

평균 절대 오차 (MAE)

$$MAE = (1/n) \sum |y_i - \hat{y}_i|$$

오차를 그대로 더함 → 이상치에 덜 민감. 0에서 미분이 안 되는 게 단점

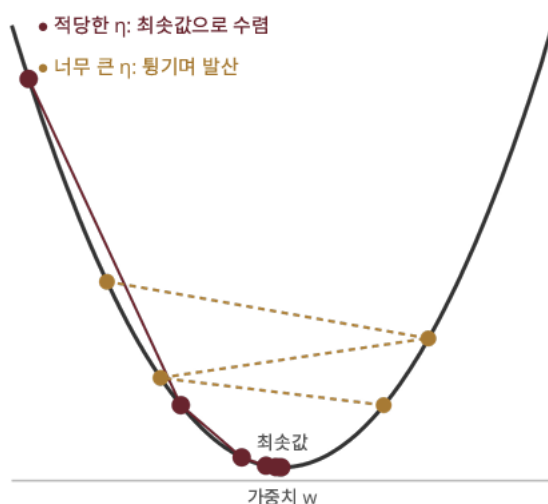
5.2 경사하강법

비용 함수의 값을 계산한 뒤에는 이 값을 감소시키는 방향으로 가중치를 수정해야 한다. 이때 사용되는 방법이 경사하강법이다. 경사하강법은 현재 가중치에서 비용 함수의 기울기를 계산하고, 기울기와 반대 방향으로 가중치를 이동시키면서 비용 함수의 최솟값을 찾는 방법이다.

가중치 갱신식은 다음과 같다.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

여기서 $\frac{\partial L}{\partial w}$ 는 가중치에 대한 손실 함수의 기울기이며, η 는 학습률을 의미한다. 기울기가 양수이면 가중치를 감소시키고, 기울기가 음수이면 가중치를 증가시키는 방향으로 이동한다.

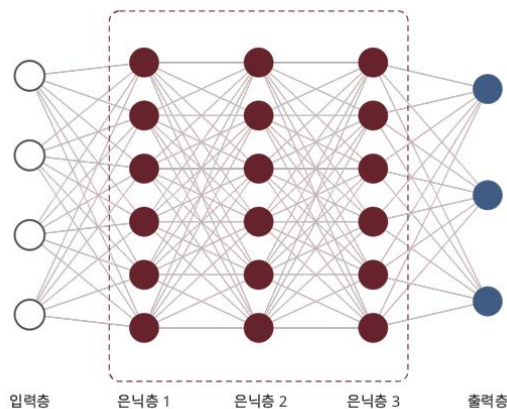


학습률은 한 번의 가중치 갱신에서 얼마나 크게 이동할지를 결정한다. 학습률이 너무 작으면 최솟값에 도달하기까지 많은 시간이 필요하고, 너무 크면 최솟값을 지나쳐 값이 계속 튀면서 학습이 불안정해지거나 발산할 수 있다. 따라서 적절한 학습률을 설정하는 것이 중요하다.

경사하강법은 한 번의 갱신에 사용하는 데이터의 양에 따라 배치 GD, 미니배치 SGD, SGD 등으로 구분할 수 있으며, 실제 학습에서는 Adam이나 Momentum을 적용한 SGD와 같은 옵티마이저도 사용된다.

6. 딥러닝 및 심층신경망

인공신경망은 퍼셉트론과 같은 뉴런을 여러 개의 층으로 연결하여 구성한 구조이다. 이때 입력을 받는 층을 입력층, 중간에서 특징을 추출하고 조합하는 층을 은닉층, 최종 결과를 출력하는 층을 출력층이라고 한다.



은닉층이 2개 이상 쌓인 신경망을 심층신경망 (DNN, Deep Neural Network)이라고 하며, 이러한 심층신경망을 학습시키는 과정을 딥러닝이라고 한다. 입력층은 픽셀 값과 같은 데이터를 받아들이고, 은닉층은 입력 데이터의 특징을 추출하고 조합하며, 출력층은 클래스 확률이나 좌표와 같은 최종 예측값을 출력한다.

모든 뉴런이 다음 층의 모든 뉴런과 연결되어 있는 구조를 완전연결층(Fully Connected Layer) 또는 Dense Layer라고 한다. 이러한 구조에서는 한 층의

각 뉴런이 다음 층의 모든 뉴런과 연결된다.

6.1 순전파

순전파는 입력층에서 시작하여 은닉층과 출력층 방향으로 값을 계산해 나가는 과정이다. 한 층에서는 이전 층의 출력값에 가중치를 곱하고 편향을 더하여 가중합을 계산한 뒤, 활성화 함수를 적용하여 다음 층으로 전달할 값을 구한다.

한 층의 계산은 다음과 같이 표현할 수 있다.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

파라미터 개수 세어보기

28×28 흑백 이미지 → 784 - 128 - 10 신경망

층	W	b	합
784 → 128	100,352	128	100,480
128 → 10	1,280	10	1,290

총 101,770개의 숫자를 학습해야 함

→ 이걸 전부 손으로 미분할 순 없으니 역전파가 필요

여기서 W 는 가중치 행렬, b 는 편향, a 는 각 층의 출력값을 의미한다. 첫 번째 층의 입력값 $a^{(0)}$ 는 입력 데이터 x 가 되며, 마지막 층에서 계산된 출력값은 모델의 예측값 $\hat{y}$ 가 된다.

신경망의 층이 많아질수록 학습해야 하는 가중치와 편향의 수 역시 크게 증가한다. 예를 들어 28×28 크기의 흑백 이미지를

입력으로 사용하고, 784개의 입력 뉴런과 128개의 은닉 뉴런, 10개의 출력 뉴런으로 구성된 신경망에서는 총 101,770개의 파라미터를 학습해야 한다.

이처럼 많은 파라미터를 효율적으로 학습하기 위해서는 각 파라미터가 손실에 얼마나 영향을 주

는지 계산해야 하며, 이를 위해 역전파 알고리즘을 사용한다.

7. 역전파 알고리즘

경사하강법을 이용하여 가중치를 수정하려면 각 가중치가 손실에 얼마나 영향을 주는지를 나타내는 기울기 $\frac{\partial L}{\partial w}$ 를 계산해야 한다. 역전파는 출력층에서 발생한 오차를 뒤에서 앞으로 전달하면서 각 가중치의 기울기를 효율적으로 계산하는 방법이다.

역전파의 핵심은 연쇄 법칙(Chain Rule)을 이용하는 것이다. 하나의 가중치에 대한 손실의 변화량은 다음과 같이 표현할 수 있다.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

즉, 손실 함수에서 가중치까지 연결된 여러 계산 과정을 각각 미분한 뒤 곱하여 최종 기울기를 구한다.

역전파가 뒤쪽 층에서 앞쪽 층으로 진행되는 이유는 뒤쪽 층에서 이미 계산한 기울기를 앞쪽 층에서 다시 사용할 수 있기 때문이다. 각 노드는 이전 단계에서 전달받은 기울기에 자신이 담당하는 연산의 미분값을 곱하여 앞쪽으로 전달한다. 이러한 방식으로 모든 계산식을 처음부터 다시 미분하지 않고도 많은 파라미터의 기울기를 효율적으로 계산할 수 있다.

왜 “역”방향?

뒤쪽 층에서 이미 구한 기울기를 앞쪽 층이 그대로 재사용한다. 그래서 파라미터가 수백만 개여도 순전파 1번 + 역전파 1번이면 모든 기울기가 나온다

각 노드는 자기 일만

노드는 들어온 기울기(upstream)에 자기의 국소 미분(local gradient)만 곱해서 앞으로 넘긴다. 전체 식을 알 필요가 없음

7.1 신경망 학습의 전체 흐름

신경망의 학습은 데이터 입력, 순전파, 손실 계산, 역전파, 가중치 갱신의 과정을 반복하면서 이루어진다.

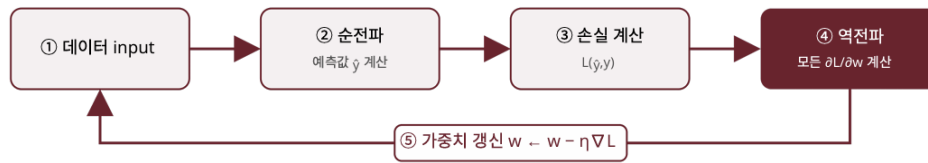
먼저 입력 데이터를 신경망에 전달하고 순전파를 수행하여 예측값 $\hat{y}$ 를 계산한다. 이후 예측값과 실제 정답 y 를 이용하여 손실 $L(\hat{y}, y)$ 을 계산한다.

계산된 손실을 이용하여 역전파를 수행하면 각 가중치에 대한 기울기 $\frac{\partial L}{\partial w}$ 를 구할 수 있다. 이후 경사하강법을 이용하여 다음과 같이 가중치를 갱신한다.

$$w \leftarrow w - \eta \nabla L$$

가중치가 갱신되면 다시 새로운 데이터를 입력하거나 같은 과정을 반복하면서 손실이 감소하는 방향으로 모델을 학습시킨다.

전체 과정은 다음과 같이 정리할 수 있다.



8. PyTorch 대응 정리

PyTorch에서는 신경망을 구성하기 위한 계층, 활성화 함수, 손실 함수, 옵티마이저, 자동 미분 기능 등을 각각의 모듈과 클래스로 제공한다. 앞에서 정리한 머신러닝 개념들은 PyTorch의 여러 함수와 클래스를 이용하여 실제 코드로 구현할 수 있다.

8.1 퍼셉트론과 nn.Linear

퍼셉트론에서 수행하는 기본 연산은 입력값에 가중치를 곱하고 편향을 더하는 것이다.

$$z = Wx + b$$

PyTorch에서는 이러한 연산을 `torch.nn.Linear` 클래스를 이용하여 구현할 수 있다. `nn.Linear`는 입력 데이터에 가중치 행렬을 곱하고 편향을 더하는 선형 변환을 수행하며, 입력 뉴런 수와 출력 뉴런 수를 지정하여 사용할 수 있다.

예를 들어 입력값이 3개이고 출력값이 1개인 퍼셉트론은 다음과 같이 구성할 수 있다.

```
import torch.nn as nn

linear = nn.Linear(3, 1)
```

여기서 3은 입력값의 개수이고 1은 출력값의 개수를 의미한다. `nn.Linear` 내부의 가중치와 편향은 학습 가능한 파라미터로 관리된다.

다만 `nn.Linear` 자체는 가중합과 편향을 계산하는 역할만 수행한다. 퍼셉트론과 같이 최종 출력을 결정하려면 이후에 활성화 함수를 추가해야 한다. 즉 개념적으로는 다음과 같이 대응시킬 수 있다.

$$\text{퍼셉트론} = \text{Linear 연산} + \text{활성화 함수}$$

Step 함수의 경우 PyTorch에서 `torch.heaviside()`를 이용하면 입력이 음수일 때 0, 양수일 때 1을 반환하는 계단 함수 형태를 구현할 수 있다.

8.2 활성화 함수

PyTorch에서는 신경망에서 자주 사용되는 활성화 함수를 `torch.nn` 모듈을 통해 제공한다.

활성화 함수	PyTorch 대응
Step	<code>torch.heaviside()</code>
Sigmoid	<code>nn.Sigmoid()</code>

Tanh	nn.Tanh()
ReLU	nn.ReLU()
Leaky ReLU	nn.LeakyReLU()

Tanh는 입력값에 하이퍼볼릭 탄젠트 함수를 적용하는 모듈이며, ReLU와 Leaky ReLU 역시 신경망의 비선형성을 추가하는 활성화 함수로 사용할 수 있다.

예를 들어 ReLU를 사용하려면 다음과 같이 작성할 수 있다.

```
relu = nn.ReLU()
output = relu(input)
```

Leaky ReLU는 음수 구간의 기울기를 negative_slope 값으로 지정할 수 있다. 앞에서 사용한

$$f(z) = \max(0.1z, z)$$

와 같은 형태를 구현하려면 다음과 같이 작성할 수 있다.

```
leaky_relu = nn.LeakyReLU(negative_slope=0.1)
```

PyTorch의 Leaky ReLU 계열 함수는 음수 입력에서 지정된 작은 기울기를 유지하도록 구성할 수 있다.

8.3 비용 함수

앞에서 다룬 MSE, MAE, Cross Entropy 등의 손실 함수 역시 PyTorch에서 클래스로 제공된다.

개념	PyTorch 클래스
MSE	nn.MSELoss()
MAE	nn.L1Loss()
Binary Cross Entropy	nn.BCELoss()
Categorical Cross Entropy	nn.CrossEntropyLoss()

다중 클래스 분류에서는 nn.CrossEntropyLoss()를 사용할 수 있으며, 이 클래스는 모델이 출력한 각 클래스의 값과 실제 정답을 이용하여 Cross Entropy 손실을 계산한다.

예를 들어 MSE 손실 함수를 사용하는 형태는 다음과 같다.

```
criterion = nn.MSELoss()
loss = criterion(output, target)
```

이때 계산된 loss 값이 앞에서 설명한 모델의 예측 오차에 해당한다.

8.4 경사하강법과 Optimizer

PyTorch에서는 가중치를 직접 식으로 수정하기보다 torch.optim 모듈의 옵티마이저를 사용하는 것이 일반적이다.

기본적인 확률적 경사하강법은 `torch.optim.SGD` 클래스로 구현할 수 있다. `lr`은 학습률을 의미하며 필요하면 `momentum` 등의 옵션을 함께 설정할 수 있다.

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.01
)
```

가중치를 실제로 갱신할 때는 다음과 같이 사용한다.

```
optimizer.step()
```

즉 앞에서 배운

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

연산을 PyTorch에서는 옵티마이저가 계산된 기울기를 이용하여 수행한다고 볼 수 있다.

8.5 역전파와 Autograd

PyTorch에서는 역전파를 직접 수식으로 구현하지 않아도 자동 미분 기능인 **Autograd**를 이용할 수 있다. Autograd는 순전파 과정에서 수행된 연산을 계산 그래프로 기록하고, 역방향으로 그래프를 따라가면서 연쇄 법칙을 이용해 기울기를 계산한다.

손실이 계산된 뒤에는 다음과 같이 `backward()`를 호출한다.

```
loss.backward()
```

`backward()`가 실행되면 학습 가능한 파라미터에 대한 기울기가 계산되고 각 파라미터의 `.grad`에 저장된다. PyTorch의 `backward()`는 계산 그래프를 연쇄 법칙에 따라 미분하여 기울기를 계산한다.

다음 학습 단계로 넘어가기 전에는 이전에 저장된 기울기를 초기화한 뒤 다시 역전파를 수행한다.

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

8.6 심층신경망

PyTorch에서는 일반적으로 `nn.Module`을 상속하여 신경망 모델을 정의한다. `nn.Module`은 모든 신경망 모듈의 기본 클래스이며, 여러 개의 계층을 하나의 모델 내부에 포함시킬 수 있다.

여러 층을 순서대로 연결하는 단순한 신경망은 `nn.Sequential`을 이용하여 구성할 수 있다. `Sequential`에 등록된 계층은 입력값을 앞에서부터 차례대로 전달한다.

예를 들어 앞에서 살펴본

784 → 128 → 10

형태의 신경망은 다음과 같이 구성할 수 있다.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
```

여기서 첫 번째 nn.Linear가 입력층과 은닉층을 연결하고, nn.ReLU가 비선형성을 추가하며, 두 번째 nn.Linear가 은닉층과 출력층을 연결한다.

9. PyTorch를 이용한 신경망 학습 예제

PyTorch에서는 신경망의 구조를 정의한 뒤 손실 함수와 옵티마이저를 설정하고, 순전파와 역전파를 반복하면서 모델을 학습시킨다. 일반적인 학습 과정은 **예측값 계산 → 손실 계산 → 기울기 초기화 → 역전파 → 가중치 갱신** 순서로 진행된다. PyTorch 공식 튜토리얼에서도

optimizer.zero_grad(), loss.backward(), optimizer.step()의 순서로 학습을 수행한다. 예를 들어 784개의 입력값을 받아 128개의 은닉 뉴런을 거친 뒤 10개의 클래스를 출력하는 신경망은 다음과 같이 구성할 수 있다.

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
```

nn.Linear(784, 128)은 입력층과 은닉층 사이의 가중치와 편향을 계산하며, nn.ReLU()는 비선형성을 추가한다. 마지막 nn.Linear(128, 10)은 은닉층의 결과를 10개의 출력값으로 변환한다.

nn.Sequential을 사용하면 이러한 계층들을 순서대로 연결할 수 있다.

분류 문제에서는 다음과 같이 손실 함수와 옵티마이저를 설정할 수 있다.

```
loss_fn = nn.CrossEntropyLoss()

optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.01
)
```

CrossEntropyLoss는 모델의 출력과 실제 클래스 사이의 손실을 계산하고, SGD는 계산된 기울기를 이용하여 모델의 가중치를 수정한다. model.parameters()를 통해 학습할 가중치와 편향이 옵티마이저에 전달된다.

실제 학습 과정은 다음과 같이 작성할 수 있다.

```
optimizer.zero_grad()

output = model(x)
loss = loss_fn(output, y)

loss.backward()
optimizer.step()
```

먼저 optimizer.zero_grad()를 이용하여 이전 학습에서 저장된 기울기를 초기화한다. PyTorch에서는 기본적으로 기울기가 누적되기 때문에 새로운 역전파를 수행하기 전에 초기화하는 과정이 필요하다.

다음으로

```
output = model(x)
```

를 실행하면 입력 데이터가 신경망의 각 층을 순서대로 통과하면서 **순전파**가 수행된다.

이후

```
loss = loss_fn(output, y)
```

를 통해 모델의 예측값과 실제 정답 사이의 손실을 계산한다.

```
loss.backward()
```

를 실행하면 역전파가 수행되어 각 가중치와 편향에 대한 기울기가 계산된다. 계산된 기울기는 각 파라미터의 .grad에 저장된다

마지막으로

```
optimizer.step()
```

을 실행하면 계산된 기울기를 이용하여 실제 가중치와 편향이 갱신된다. SGD.step()은 한 번의 최적화 단계를 수행하는 메서드이다.

따라서 PyTorch의 학습 과정은 다음과 같이 정리할 수 있다.

입력 데이터 → 순전파 → 손실 계산 → 역전파 → 가중치 갱신

이 과정은 앞에서 정리한 신경망의 전체 학습 과정과 동일하며, PyTorch에서는 각 과정을 함수와 클래스를 이용하여 비교적 간단하게 구현할 수 있다.

10. 학습에 사용되는 주요 설정값

신경망을 학습할 때는 모델의 구조뿐만 아니라 데이터를 몇 번 반복해서 학습할지, 한 번에 몇 개의 데이터를 사용할지, 가중치를 어느 정도 크기로 수정할지 등을 설정해야 한다. 이러한 값들은 학습 과정과 결과에 영향을 주며 대표적으로 Epoch, Batch Size, Learning Rate 등이 있다.

10.1 Epoch

Epoch는 전체 학습 데이터를 한 번 모두 사용하여 학습한 것을 의미한다. 예를 들어 학습 데이터가 1,000개라면 1,000개의 데이터를 모두 사용하여 한 차례 학습을 완료했을 때 1 Epoch가 진행된 것이다.

일반적으로 신경망은 한 번의 Epoch만으로 충분히 학습되지 않기 때문에 여러 Epoch 동안 동일한 학습 데이터를 반복해서 사용한다. 각 Epoch에서는 순전파를 통해 예측값을 계산하고, 손실을 구한 뒤 역전파와 가중치 갱신을 반복한다.

예를 들어 PyTorch에서는 다음과 같이 Epoch 수만큼 반복하여 학습할 수 있다.

```
epochs = 10

for epoch in range(epochs):
    # 학습 과정
    pass
```

즉 epochs = 10은 전체 학습 데이터를 총 10번 반복하여 학습한다는 의미이다.

10.2 Batch와 Batch Size

전체 학습 데이터를 한 번에 모두 사용하지 않고 여러 개의 작은 묶음으로 나누어 학습할 수 있는데, 이러한 데이터 묶음을 Batch라고 한다. Batch Size는 하나의 Batch에 포함되는 데이터의 개수를 의미한다.

예를 들어 데이터가 총 1,000개이고 Batch Size가 100이라면 한 Epoch는 10개의 Batch로 나뉘게 된다.

$$\frac{1000}{100} = 10$$

따라서 한 Epoch 동안 약 10번의 학습 단계가 진행된다.

PyTorch에서는 DataLoader의 batch_size를 이용하여 Batch Size를 설정할 수 있다. DataLoader는 데이터를 자동으로 Batch 단위로 구성하는 기능을 제공한다.

```
train_loader = DataLoader(
    training_data,
    batch_size=64,
    shuffle=True
)
```

위 코드에서는 학습 데이터를 한 번에 64개씩 가져와 학습한다. `shuffle=True`를 사용하면 Epoch 마다 데이터의 순서를 섞어 사용할 수 있다. PyTorch의 `DataLoader`는 자동 batching, 데이터 순서 설정 등의 기능을 지원한다.

10.3 Learning Rate

Learning Rate는 경사하강법을 이용하여 가중치를 갱신할 때 한 번에 얼마나 크게 이동할지를 결정하는 값이다.

앞에서 살펴본 가중치 갱신식에서 η 가 Learning Rate에 해당한다.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Learning Rate가 너무 작으면 가중치가 조금씩만 변하기 때문에 학습에 많은 시간이 필요할 수 있다. 반대로 너무 크면 비용 함수의 최솟값을 지나치면서 학습이 불안정해질 수 있다. 따라서 모델과 데이터에 적절한 Learning Rate를 설정하는 것이 중요하다.

PyTorch에서는 옵티마이저를 생성할 때 `lr`을 이용하여 Learning Rate를 설정한다.

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.01
)
```

여기서 `lr=0.01`은 가중치를 갱신할 때 사용할 Learning Rate를 의미한다.

10.4 Optimizer

Optimizer는 역전파를 통해 계산된 기울기를 이용하여 모델의 가중치와 편향을 어떻게 수정할지를 결정하는 방법이다. PyTorch에서는 `torch.optim` 모듈을 통해 여러 종류의 최적화 알고리즘을 제공한다.

가장 기본적인 방법 중 하나는 SGD(Stochastic Gradient Descent)이다.

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.01
)
```

SGD는 계산된 기울기와 Learning Rate를 이용하여 손실이 감소하는 방향으로 파라미터를 수정한다. 필요에 따라 Momentum을 추가하여 사용할 수도 있다.

또 다른 대표적인 Optimizer로 Adam이 있다.

```
optimizer = torch.optim.Adam(  
    model.parameters(),  
    lr=0.001  
)
```

Adam은 기울기의 이동 평균과 기울기 제곱의 이동 평균을 이용하여 각 파라미터의 갱신 크기를 조절하는 방식이다. PyTorch에서는 torch.optim.Adam으로 사용할 수 있다.

실제 학습에서는 Optimizer를 생성한 뒤 다음과 같은 순서로 사용한다.

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

zero_grad()는 이전 기울기를 초기화하고, backward()는 역전파를 통해 새로운 기울기를 계산하며, step()은 계산된 기울기를 이용하여 실제 모델 파라미터를 갱신한다.

11. 머신러닝 개념과 PyTorch 대응 정리

앞에서 살펴본 머신러닝과 신경망의 주요 개념은 PyTorch의 여러 클래스와 함수를 이용하여 구현할 수 있다. 개념과 PyTorch에서의 대응 관계를 정리하면 다음과 같다.

머신러닝 개념	PyTorch 대응
퍼셉트론의 가중합	nn.Linear()
Step 함수	torch.heaviside()
Sigmoid	nn.Sigmoid()
Tanh	nn.Tanh()
ReLU	nn.ReLU()
Leaky ReLU	nn.LeakyReLU()
MSE	nn.MSELoss()
MAE	nn.L1Loss()
Binary Cross Entropy	nn.BCELoss()
Cross Entropy	nn.CrossEntropyLoss()
경사하강법	torch.optim.SGD()
Adam Optimizer	torch.optim.Adam()
역전파	loss.backward()
기울기 초기화	optimizer.zero_grad()
가중치 갱신	optimizer.step()
심층신경망 구성	nn.Module, nn.Sequential
완전연결층	nn.Linear()

퍼셉트론에서 수행하는 $Wx + b$ 연산은 `nn.Linear()`를 이용하여 구현할 수 있으며, 그 뒤에 `nn.ReLU()`나 `nn.Sigmoid()`와 같은 활성화 함수를 연결하여 비선형성을 추가할 수 있다.

모델의 예측 결과와 실제 정답의 차이는 `nn.MSELoss()`, `nn.L1Loss()`, `nn.CrossEntropyLoss()` 등의 손실 함수를 이용하여 계산한다. 계산된 손실에 대해 `loss.backward()`를 실행하면 역전파가 수행되어 각 파라미터의 기울기가 계산된다.

이후 `torch.optim.SGD()`나 `torch.optim.Adam()`과 같은 `Optimizer`를 이용하여 계산된 기울기를 바탕으로 가중치와 편향을 갱신한다. 이러한 과정은 앞에서 살펴본 **순전파 → 손실 계산 → 역전파 → 가중치 갱신**의 신경망 학습 과정과 연결된다.

간단히 코드의 흐름으로 나타내면 다음과 같다.

```
optimizer.zero_grad()

output = model(x)
loss = loss_fn(output, y)

loss.backward()
optimizer.step()
```

`model(x)`에서는 순전파가 수행되고, `loss_fn()`을 통해 손실을 계산한다. 이후 `loss.backward()`를 실행하여 역전파로 기울기를 구하고, `optimizer.step()`을 통해 실제 모델의 파라미터를 갱신한다.

12. 결론 및 느낀점

이번 과제를 통해 머신러닝의 기본 개념부터 퍼셉트론, 활성화 함수, 비용 함수, 경사하강법, 심층 신경망, 역전파까지 신경망이 학습되는 전체 과정을 순서대로 정리할 수 있었다. 처음에는 각각의 개념이 따로 존재하는 것처럼 느껴졌지만, 내용을 정리하면서 각 개념이 하나의 학습 과정 안에서 서로 연결되어 있다는 것을 알 수 있었다.

퍼셉트론에서는 입력값에 가중치를 곱하고 편향을 더한 뒤 활성화 함수를 통해 출력값을 결정하며, 이 과정에서 적절한 가중치와 편향을 찾는 것이 학습의 중요한 부분이라는 것을 이해하였다. 또한 활성화 함수가 신경망에 비선형성을 추가하고, 비용 함수가 모델의 예측이 정답과 얼마나 차이가 나는지를 나타낸다는 점을 확인하였다.

비용 함수의 값을 줄이기 위해 경사하강법을 사용하고, 역전파를 통해 각 가중치에 대한 기울기를 계산한다는 과정도 이해할 수 있었다. 특히 순전파를 통해 예측값을 계산하고, 손실을 구한 뒤 역전파와 가중치 갱신을 반복하는 과정이 신경망 학습의 기본 흐름이라는 점을 알 수 있었다.

또한 각 개념이 PyTorch에서는 어떤 함수와 클래스로 구현되는지를 함께 정리하면서 이론적인 개념이 실제 코드에서 어떻게 사용되는지 연결해서 이해할 수 있었다. `nn.Linear()`, 활성화 함수, 손실 함수, `Optimizer`, `backward()`와 같은 기능들이 각각 앞에서 학습한 개념과 연결된다는 점을 확인하였다.

이번 과제를 통해 단순히 함수의 사용 방법만 확인하는 것이 아니라, 각 함수가 신경망 학습 과

정에서 어떤 역할을 하는지 이해하는 것이 중요하다고 느꼈다. 앞으로 실제 머신러닝 모델을 구현할 때에도 코드만 사용하는 것이 아니라, 내부에서 어떤 계산과 학습 과정이 이루어지는지를 함께 생각하며 학습하고자 한다.